

Technical Report — Disaster Event Intelligence API

Module: COMP3011

Assessment: Coursework 1

Name: Hao Ji

1. Introduction

This project is a backend API for storing, querying, and analysing disaster events. The system was designed as a SQL-backed FastAPI application that supports local event management, external data ingestion, metadata tracking, and analytical queries.

The project goes beyond a minimal CRUD implementation by:

- supporting multiple disaster event types rather than a single narrow entity
- integrating external public data sources
- storing source metadata and ingestion history
- providing analytics and risk assessment endpoints over locally stored data

The resulting system is not only able to ingest data, but also to treat that data as part of an independent local application domain.

2. Problem Definition

The coursework requires a SQL-backed CRUD API with multiple endpoints and proper error handling. This project extends that requirement into a disaster intelligence system with ingestion and analytics capabilities.

3. Technology Stack and Justification

- Python was chosen for flexibility and ecosystem support.
- FastAPI provides lightweight API development and automatic documentation.
- SQLAlchemy enables structured database interaction.
- SQLite simplifies local deployment.
- HTTPX supports external API integration.
- Pytest ensures correctness through testing.

4. System Design

The system follows a layered architecture including route, dependency, CRUD, model, schema, and database layers. The domain includes three entities: `disaster_events`, `source_metadata` and `ingest_runs`.

5. API Design

5.1 Events

The Events module provides the core CRUD functionality, This is the centre of the application and satisfies the coursework CRUD requirement.

- create events
- list events
- retrieve by ID
- replace records
- patch records
- delete records

5.2 Ingest

The Ingest module imports data from: NASA EONET for wildfires & USGS Earthquake Catalog for earthquakes. These external APIs are treated as data suppliers rather than as the system's live backing store. The ingested data is transformed and persisted locally.

5.3 Analytics

The Analytics module provides: summary statistics, daily timeseries, spatial hotspots and risk assessment. This elevates the project beyond basic CRUD by providing interpretation and intelligence over the dataset.

5.4 Health

The Health module provides: service health status & database connectivity checks
These endpoints are intentionally public so that deployment verification and monitoring can happen without authentication barriers.

6. Authentication Design

The system implements HTTP Basic Authentication on the backend. This is enforced via a FastAPI dependency that:

- reads the Authorization header
- extracts Basic Auth credentials
- validates them against configured values
- returns 401 Unauthorized if validation fails

Basic Authentication was chosen because it is simple, standards-based, easy to document, and sufficient for coursework demonstration. If this system were extended further, a more production-ready choice would be token-based authentication or role-based access control.

7. External Data Integration

7.1 NASA EONET: Used for wildfire event ingestion. The API data is: previewed through a non-persistent endpoint and synchronized through a persistent ingestion endpoint.

7.2 USGS Earthquake CatLog: Used for earthquake event ingestion. The API data is transformed into the local disaster event format and then stored.

7.3 Why Local Persistence Matters: The project does not merely proxy upstream APIs. Instead, it persists the data in a local SQL database and performs analytics on local records. This design is more aligned with coursework requirement to build a real API system rather than a thin wrapper over third-party services.

8. Error Handling

The API uses conventional HTTP status codes:

- 200 OK
- 201 Created
- 204 No Content
- 401 Unauthorized
- 404 Not Found
- 422 Unprocessable Entity
- 502 Bad Gateway
- 503 Service Unavailable

9. Testing Approach

The test suite was designed to cover both success and failure scenarios. Covered areas include:

- CRUD creation and filtered reads
- authentication failures
- invalid payloads
- boundary values
- duplicate ingestion handling
- upstream provider failures
- public health checks
- database failure handling
- analytics correctness under controlled fixtures
- risk assessment ranking logic

10. Deployment

The application is deployable locally and has also been hosted remotely. The model uses:

- FastAPI as an ASGI application
- uvicorn as the application server
- PythonAnywhere hosting for remote availability

This allows both local testing and live demonstration through /docs.

11. Challenges Encountered

Domain Refactoring: The initial model design was too narrow once earthquake support was introduced. Refactoring to disaster_events improved extensibility and consistency, but required coordinated updates across routes, schemas, and documentation.

External API Variability: External APIs do not return fully consistent data. Records may contain missing fields, varying formats, or irregular geometries, requiring robust transformation logic and reliable duplicate detection during ingestion.

Documentation Quality: Although FastAPI provides automatic OpenAPI documentation, a coursework-ready submission required more structured and curated materials. This led to the development of additional documentation, including API explanations and system design descriptions.

Deployment Practicalities: Deploying the ASGI application on PythonAnywhere introduced platform-specific considerations, such as environment configuration, startup commands, and service reload workflows.

12. Limitations

The current system has several limitations:

- SQLite is suitable for coursework but not ideal for larger-scale production systems
- authentication is intentionally basic and does not support roles or tokens
- external ingestion depends on the availability and structure of third-party APIs
- ingest runs and events are logically related, but not linked through a direct foreign key

13. Future Improvements

Future work includes PostgreSQL migration, scheduled ingestion, improved security, and advanced analytics.

14. Reflection on Design Choices

The final design reflects a conscious shift from a minimal CRUD API to a more complete backend application. The introduction of source metadata, ingestion history, analytics, and risk assessment makes the project easier to justify as a cohesive system rather than a collection of independent endpoints.

15. Generative AI Declaration

Generative AI tools were used during the development of this coursework in a declared and methodical way. AI was used for:

- planning implementation steps
- discussing design alternatives
- explaining external API documentation
- suggesting test scenarios
- refining written documentation

AI was not used as an unquestioned source of truth. All generated suggestions were reviewed, adapted, and integrated manually into the final implementation. The main benefit of AI use in this project was acceleration of exploration and structured reasoning, especially when comparing architectural options and improving testing scope. This use aligns more closely with a methodological and design-support role rather than arbitrary code generation.

16. Conclusion

This project satisfies the coursework's core technical requirements while also demonstrating a more developed system design than a minimal CRUD submission. The final application provides:

- SQL-backed CRUD functionality
- external data ingestion
- source metadata management
- ingestion audit history
- analytics and risk assessment
- authentication
- testing
- deployment support

Overall, the project represents a structured and extensible disaster event intelligence API that is suitable for coursework demonstration and discussion.

References

1. FastAPI. Available at: <https://fastapi.tiangolo.com/> [Accessed 19 March 2026].
2. SQLAlchemy. Available at: <https://docs.sqlalchemy.org/> [Accessed 19 March 2026].
3. Pydantic. Available at: <https://docs.pydantic.dev/> [Accessed 19 March 2026].
4. HTTPX. Available at: <https://www.python-httpx.org/> [Accessed 19 March 2026].
5. NASA EONET Version 3 Documentation. Available at: <https://eonet.gsfc.nasa.gov/docs/v3> [Accessed 19 March 2026].
6. USGS Earthquake Catalog API Documentation. Available at: <https://earthquake.usgs.gov/fdsnws/event/1/> [Accessed 19 March 2026].
7. PythonAnywhere Help. Available at: <https://help.pythonanywhere.com/> [Accessed 19 March 2026].

Appendix A: Generative AI Usage Logs

Example 1: System Architecture Design

Purpose: To explore how to structure the FastAPI application and separate concerns between routing, business logic, and data storage.

Excerpt: User: How should I structure a FastAPI project with ingestion and analytics features?

AI: Suggested using a layered architecture with routes, dependencies, CRUD logic, models, and schemas, and separating ingestion from core data storage.

Reflection: The suggested layered structure was adopted and extended to include a clear separation between `disaster_events`, `source_metadata`, and `ingest_runs`.

This improved the system from a simple CRUD API into a more modular and extensible architecture with better support for analytics and ingestion workflows.

Example 2: External API Integration Design

Purpose: To determine how to integrate NASA EONET and USGS APIs into the system.

Excerpt: User: Should I query external APIs directly or store the data locally?

AI: Suggested that external APIs should be treated as data sources and data should be persisted locally to support analytics and stability.

Reflection: The final design follows a local persistence approach rather than direct proxying.

External APIs are used only during ingestion, and all analytics are performed on locally stored data. This decision improves reliability, reduces dependency on upstream availability, and enables richer analysis.

Example 3: Testing Strategy and Edge Cases

Purpose: To identify meaningful test scenarios beyond basic success cases.

Excerpt: User: What test cases should I include for a FastAPI ingestion system?

AI: Suggested covering authentication failures, invalid payloads, duplicate ingestion, and upstream API errors.

Reflection: These suggestions were incorporated into the test suite, which now includes validation errors, authentication checks, ingestion edge cases, and analytics correctness.

This ensured the system behaves predictably under both normal and failure conditions.